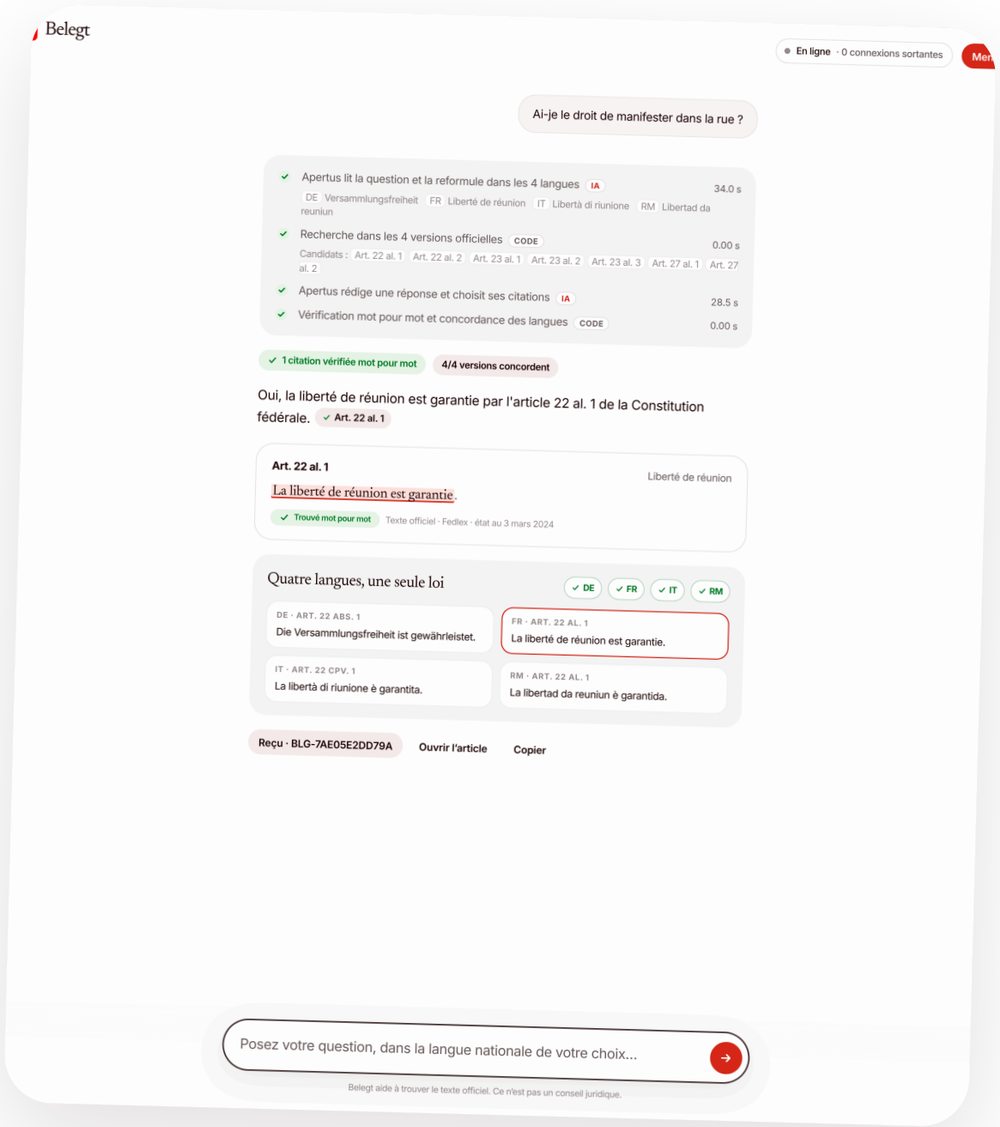
# Belegt.

## Cite, or stay silent.

A Swiss-law assistant built on **Apertus** that answers only when it can prove every claim with the official text — in German, French, Italian and Romansh, fully offline.



THE PROBLEM

# A confident wrong answer cannot serve the state.

Apertus 1.5 8B, asked alone — real output

«Le Conseil fédéral compte sept membres. Cette composition est définie à l'~~article 134~~ de la Constitution fédérale.»

× wrong It is Art. 175.

Same model, same setting — real output

«Les langues nationales de la Suisse sont l'allemand, le français et l'italien.»

× incomplete Romansh is forgotten (Art. 4).

**50%**

of Apertus-alone answers name a wrong or invented article (4/8)

## Inequality

Four official languages: an answer that changes with the language is unequal treatment before the law.

## Dependency

Sending citizens' questions to a foreign service is a sovereignty and data-protection problem first.

THE IDEA

# One rule everyone understands: cite, or stay silent.

Belegt asserts nothing it cannot point to in the official text. If the sentences don’t exist, the answer isn’t given.

**01**

### Proven

Exact sentences of the law, highlighted, one click away.

**02**

### Honest

No evidence, no answer: it abstains, explains, and shows the closest articles.

**03**

### One law, four languages

DE · FR · IT · RM checked against each other; divergences are flagged.

**04**

### A receipt

Model hash, law version, evidence, checks, network state. Fileable.

HOW IT WORKS

# The AI understands and drafts. Plain code decides.

AI · Apertus

## Understand

Rewrites the question in DE/FR/IT/RM with the Constitution's own words ("demonstrate" → "freedom of assembly").

Code

## Search

BM25 on words + character 5-grams over 657 paragraphs × 4 languages; rank fusion; article-level context.

AI · Apertus

## Draft

1–3 sentences + quotes, JSON constrained by a grammar. May answer "nothing answers this".

Code · no AI

## Verify

Deterministic rules below. Removes what is unproven. Abstains if nothing survives.

Code

## Receipt

SHA-256 of model, law text and quotes; checks; network state; duration.

**R1**
The cited paragraph exists in the official text.

**R2**
The quote appears word for word.

**R3**
The quote is long enough to mean something.

**R4**
No article number in the answer that isn't proven.

**R5**
No surviving evidence ⇒ abstain.

**X**
Independent search in each language reaches the same article.

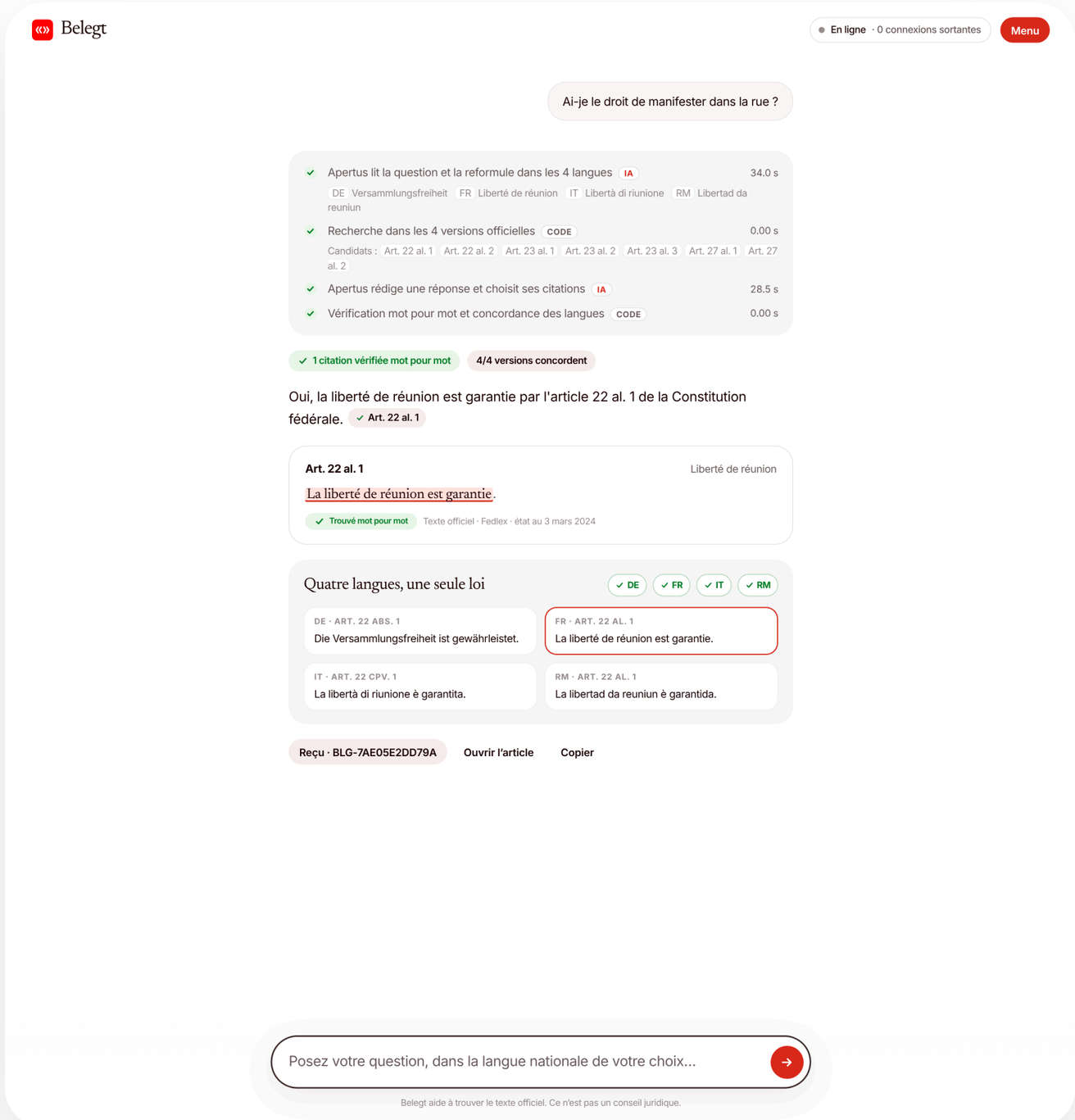
Languages are aligned by the official **Fedlex identifiers** (`art_22/para_1` is the same provision in every version): the cross-language check needs no machine translation.

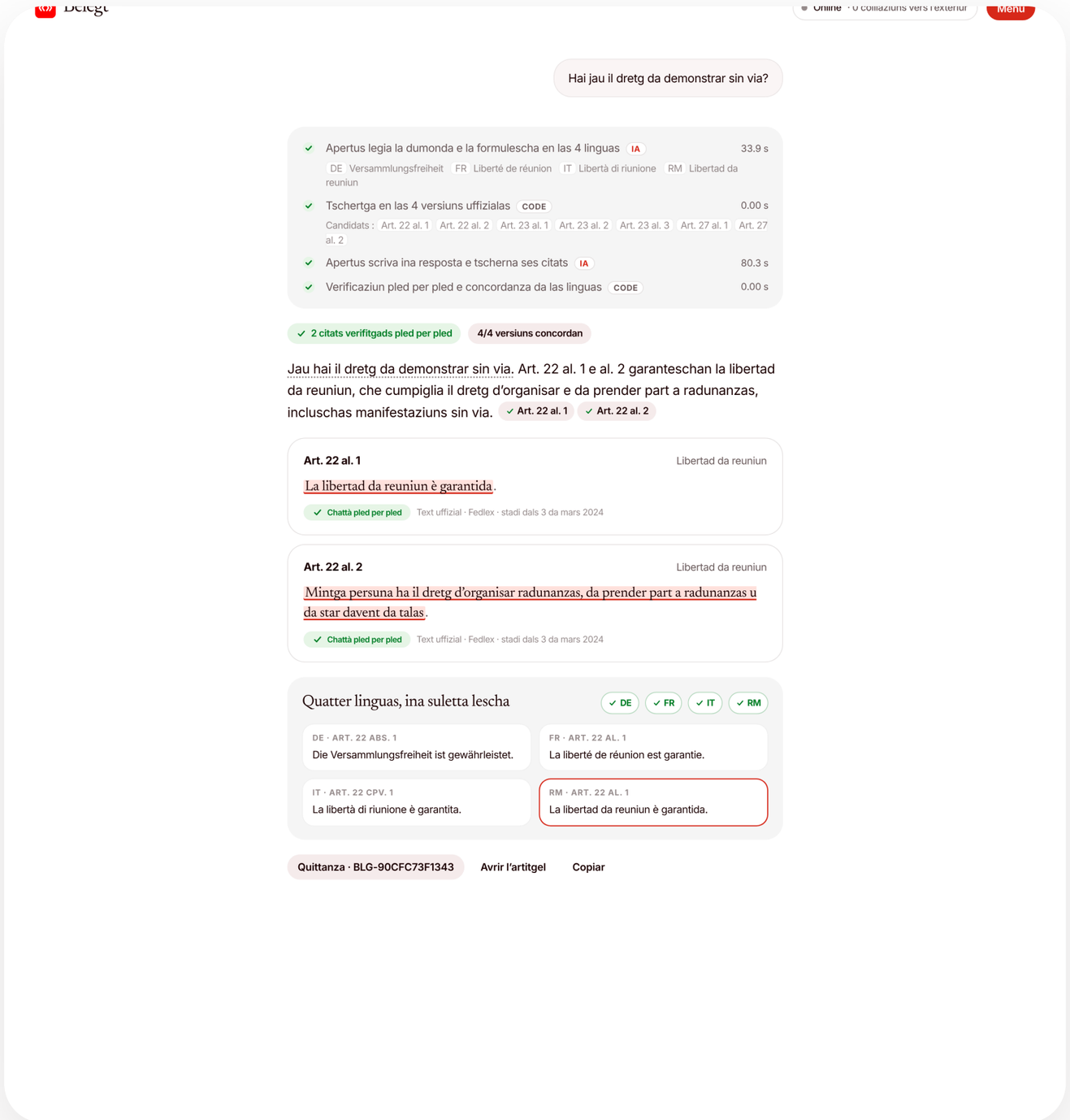
SCENE 1 · THE CLEAR QUESTION

# “Can I demonstrate in the street?”

A short answer, and next to it the exact sentences of Art. 22, found word for word in the official text. One click opens the article with the sentence highlighted.

Sentences that rephrase instead of quoting are dotted-underlined.






SCENE 2 · FOUR LANGUAGES, ONE LAW

# The same question, in Romansh.

Answer in Rumantsch, quotes from the official Romansh text, and proof that DE, FR, IT and RM lead to the same provision.

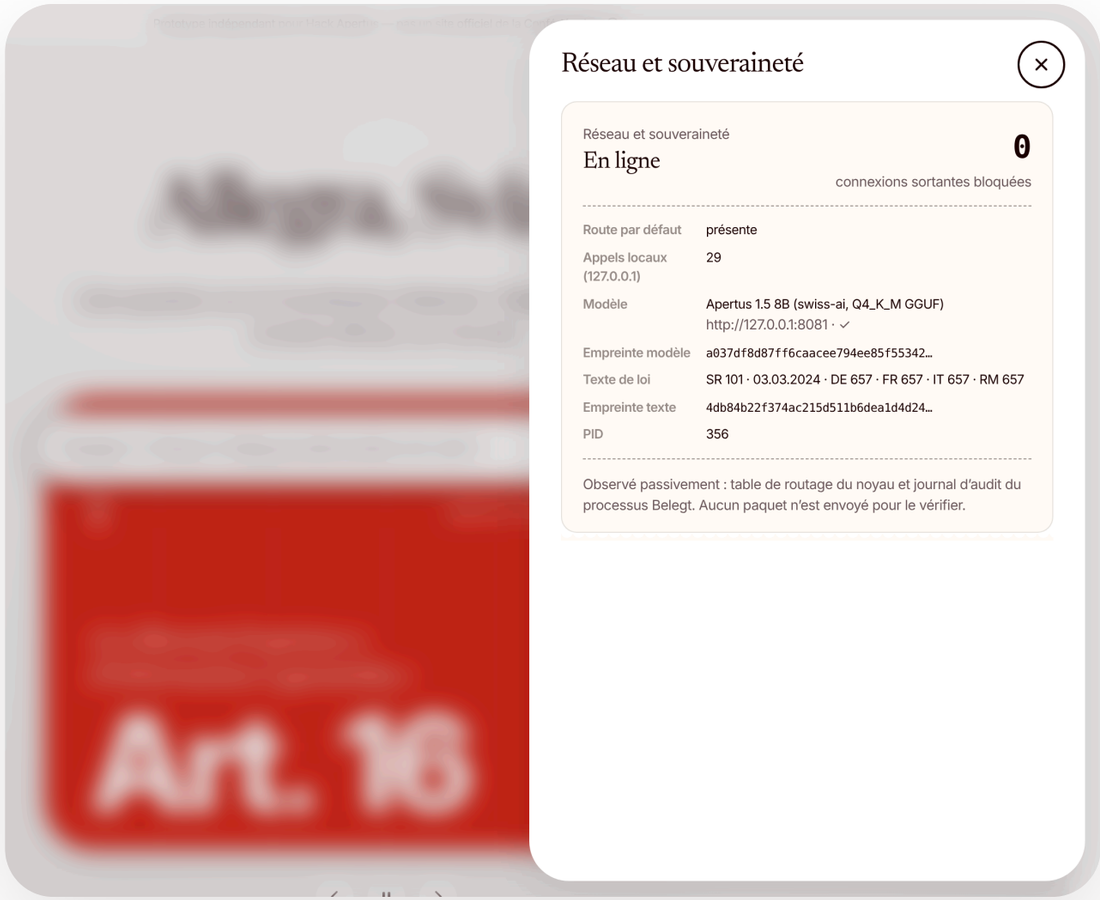
**0%**

of questions asked in all four languages get the same answer with Belegt (0/0), vs 0% for Apertus alone.

SOVEREIGN, DEMONSTRATED

# Cable unplugged. Zero outbound connections.

- **Apertus runs on the machine** — llama.cpp, CPU only, compiled without network support. Model file pinned by SHA-256.
- **Belegt blocks and counts** every outbound connection and DNS lookup with a Python audit hook. The counter is in the UI and in every receipt.
- **The demo runs in a Linux network namespace** with no interface but loopback: no route out exists at all (`scripts/offline_demo.sh`).
- **No CDN, no tracker**: fonts and scripts are served by Belegt itself. Nothing is stored server-side.

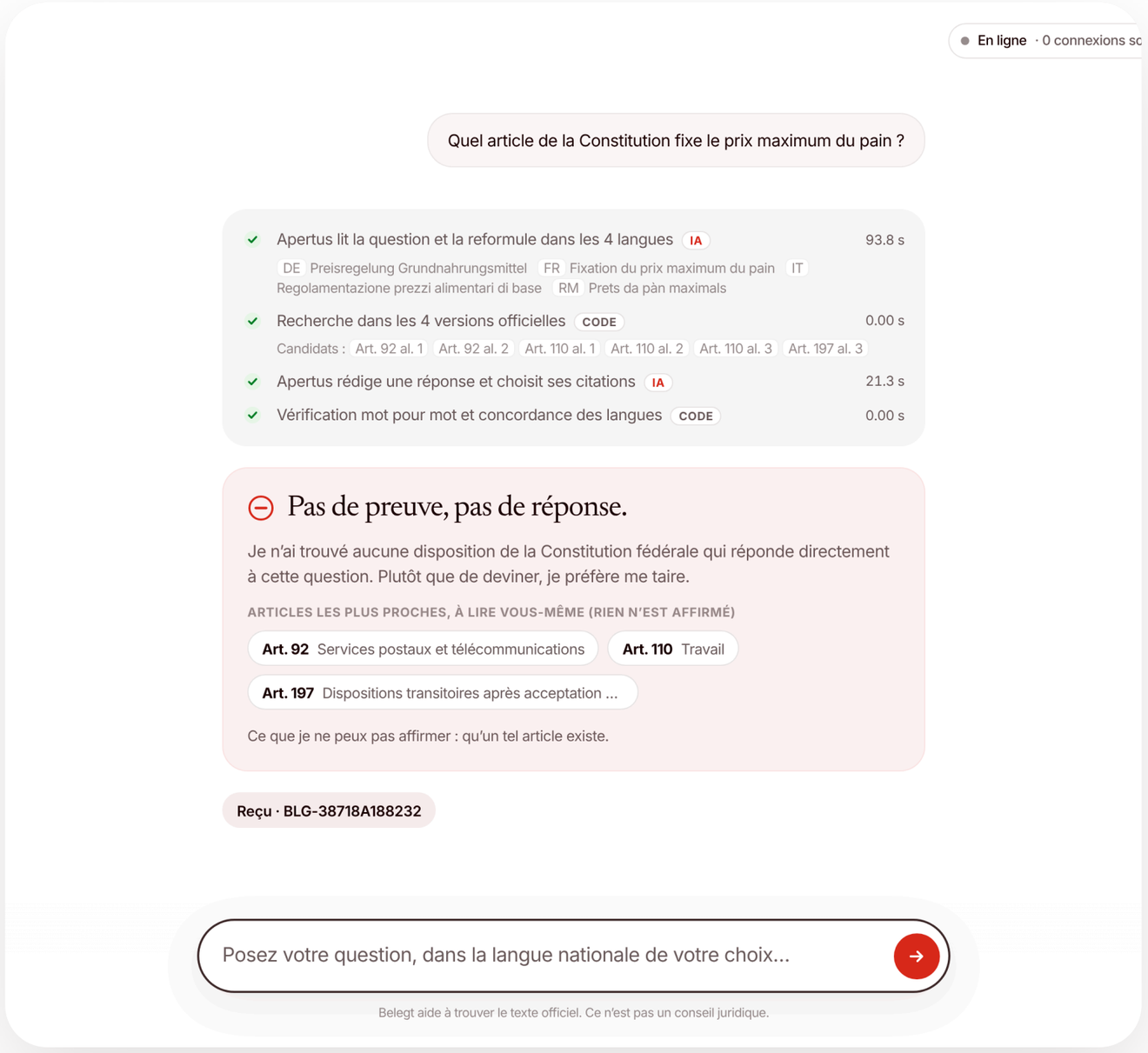


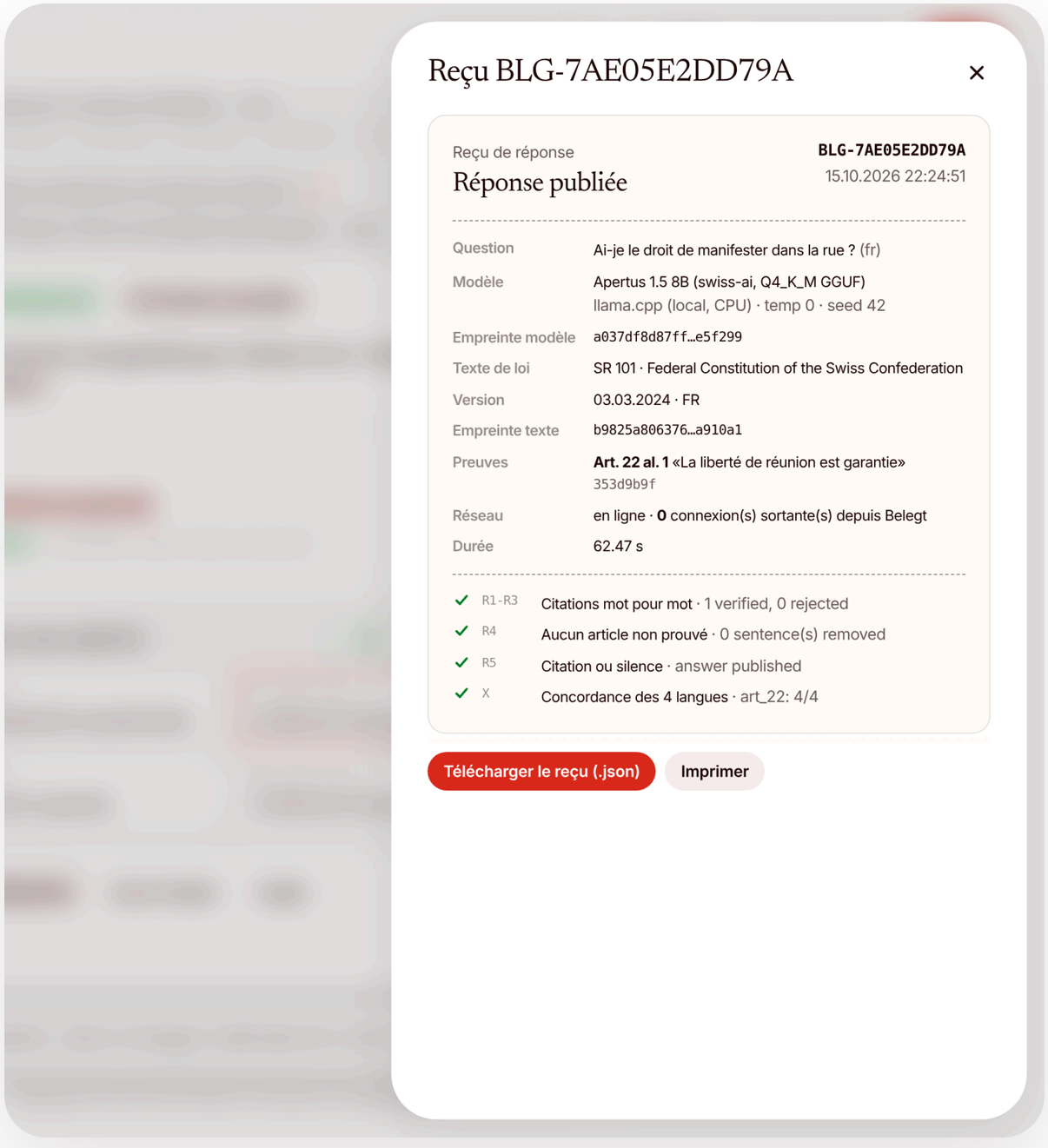
SCENE 3 · THE TRAP

# “Which article sets the maximum price of bread?”

None does. Belegt says so, points to the closest articles for the reader to check (food security, competition policy), and claims nothing.

Abstention is the expected behaviour, measured and shown — not hidden.






SCENE 4 · THE AUDITOR

# A receipt for every answer.

- **Model** name, runtime, temperature, seed, SHA-256 of the weights
- **Law** SR 101, version 03.03.2024, SHA-256 of the text
- **Evidence** each quote with its hash and character span
- **Checks** R1–R5 and cross-language agreement
- **Network** offline state and outbound-connection count

Downloadable as JSON, printable. It turns a demo into an infrastructure component.

MEASURED, NOT PROMISED

# Same model. Same questions. With and without Belegt.

| | APERTUS ALONE | APERTUS + BELEGT |
|---|---|---|
| Cites a correct article (answerable) | 50% 50% [22–78] | **88%** 88% [53–98] |
| Publishes a wrong or invented article | 50% 50% [22–78] | **12%** 12% [2–47] |
| Trap questions refused | 0% 0% [0–0] | **0%** 0% [0–0] |
| Same answer in all 4 languages | 0% 0% [0–0] | **0%** 0% [0–0] |
| Abstains although an answer exists | — | **0%** 0% [0–32] |

8 items: 8 answerable, 0 traps; parallel set asked in DE/FR/IT/RM; held-out questions written after the last design change. Apertus 1.5 8B Q4_K_M, local, temperature 0. 95% Wilson intervals. Baseline trap answers labelled by hand, with justification. Raw outputs: `eval/`.

## What the verifier caught

0 quotes not found word for word

0 sentences naming an unproven article

0 drafts with no surviving evidence

Median latency 71.0 s per answer on 4 CPU cores.

TRACK 2B CRITERIA

# Built against the five published criteria.

| CRITERION | HOW BELEGT ANSWERS | PROOF TO CHECK |
|---|---|---|
| **Meaningful use of AI** | Apertus understands, rewrites in four languages and drafts. Verification is deterministic. | “AI / Code” chips on every step; trap refusal |
| **Technical rigour** | Baseline vs Belegt on the same model, Wilson intervals, held-out set, failures kept visible. | `eval/results.md`, raw runs |
| **Value, cost, scale** | 8B model, 4-bit, CPU. ≈ CHF 3.0 per 1,000 answers on a 4-vCPU VM (assumption: CHF 0.15/h). | Latency in every receipt |
| **Sovereign deployability** | Offline, local weights, no external service, outbound audit, receipts. | `offline_demo.sh` + network panel |
| **Implementation feasibility** | Deliberately small scope that fully works; Python stdlib + llama.cpp; one command to run. | Jury can run it in 10 minutes |

WHO IT IS FOR

# A blueprint others can take tomorrow.

### Citizens & schools

Understand rights with the source one click away; teach critical thinking about AI with an AI that shows its evidence.

### Administrations

Communes, cantons, Confederation: deployable without invented content or foreign dependency; receipts for the file.

### Regulated firms

Banks, insurers, health: the same rule on their own texts — compliance needs traceability, not eloquence.

### Next corpora (the method is the product)

Civil Code (SR 210) · Data Protection Act (SR 235.1) · AVS/AHV (SR 831.10) · municipal regulations. Any Akoma Ntoso text from Fedlex drops in: `belegt/corpus.py` parses the official XML and keeps the official IDs.

WHAT BELEGT GIVES BACK TO APERTUS

# A contribution, not just a demo.

- **A weakness map**: where Apertus cites correctly, invents, or diverges across languages — with examples.
- **A reusable test set**: questions, traps, the same questions in DE/FR/IT/RM, gold articles.
- **A behaviour worth training**: cite or abstain, as a target for future alignment work.
- **A receipt format** other projects can adopt. Everything Apache-2.0.

## Limits, said first

Not legal advice. The verifier proves quotes exist — not that the paraphrase is right, nor that the quoted article is the relevant one (those cases are counted as errors). Lexical search can miss the article → abstention. ~1 min per answer on CPU. Romansh test questions written by the team, not native speakers.

“Apertus was built open so that trust could be verified. Belegt is a first public demonstration.”

**RUN IT**

```
scripts/start.sh
sudo scripts/offline_demo.sh
```

**CODE & DATA**

github.com/chessoren/swiss
Apache-2.0

**BUILT WITH**

Apertus 1.5 8B · llama.cpp · Fedlex · Python stdlib

## Prouver, ou se taire. — Cite, or stay silent.